Disability Services
University of Warwick
Senate House
Coventry
CV4 8UW

Considerate Marking cover sheet

DO NOT COPY

Student ID Number: 5714960

Note to student:

It is the responsibility of the student to add this coversheet to their coursework or examinations. University staff will be unable to attach this to your work, on your behalf.

Note to assessors/markers:

This cover sheet is to prompt assessors/markers to the Specific Learning Difference (such as Dyslexia, Dyspraxia/DCD, dysgraphia, dyscalculia), and/or other condition that this student has, which is supported through the university's Reasonable Adjustments process.

Academic staff are encouraged to not unduly penalise poor spelling, punctuation, grammar or structure, unless these form explicit assessed course criteria or competency standards, but rather mark for content and demonstrated understanding.

Further information and guidance can be found here: https://warwick.ac.uk/services/wss/students/disability/how-can-we-help/reasonable-adjustments/marking/

Kind regards

Disability Services

Date issued: 02/09/2025

CS918: Natural Language Processing Assignment

1st Saatvik Kambhampati
Department of Computer Science)
University of Warwick
Coventry, United Kingdom
Saatvik.Kambhampati@warwick.ac.uk

***Abstract*—This report presents three sentiment classifiers for Twitter data using the SemEval 2017 Task 4 dataset: a linear Support Vector Machine, a Logistic Regression classifier, and an LSTM neural network with frozen GloVe embeddings. The traditional classifiers achieved macroaveraged F1 scores of approximately 0.60 to 0.64 across three test sets, while the LSTM achieved 0.55 to 0.60. Error analysis reveals that misclassification is concentrated at adjacent sentiment boundaries, compounded by class imbalance and domain mismatch between the training representations and informal Twitter language.**

I. Introduction

Sentiment Analysis (SA) is a sub-field of natural language processing. The sub-field concerns itself with the analysis of the expressions, vocabulary, and sentence structure of an input text in an attempt to determine the sentiment of the user who created the input text [1]. The area SA is growing in importance due to its applications for a wide range of people and organisations. For example, in business analytics, reviews are analysed for opinions and sentiments. These would then help the organisation better understand their customers, thereby allowing them to improve marketing and grow sales [2]. It is increasingly important to understand the current methods available in this field so that these methods can be applied in order to extract the maximum amount of information from users' texts.

This report details the sentiment analysis of tweets from Twitter (now X) using various machine learning models. **Section II** describes the dataset and preprocessing pipeline, including noisy data handling (**Section II-B**), lemmatisation (**Section II-C**), and vector embeddings (**Section II-E**), with justifications for each design choice. **Section III** presents the three classifiers developed, and **Section V** reports and comparatively analyses their performance across all test sets.

II. Dataset & Data Preprocessing

The dataset used was the 'SemEval-2017 Task 4' Twitter dataset, from the SemEval 2017 workshop [3]. It is made of five separate files – 'twitter-training-data.txt' and 'twitter-test[1-3].txt'. Each dataset file has three columns, the ID of the tweet, its sentiment (positive, neutral, and negative), and finally, the tweet as a string.

The training dataset had 45025 tweets, whilst the three testing sets had 3531, 1853, and 2379 tweets each. Further, in the training set, 15960 were of the 'positive' sentiment, 20748 of the 'neutral' sentiment, and 8317 of the 'negative' sentiment. Thus, the models' training data was composed of 35 % positive tweets, 46 % neutral tweets, and a small minority of negative tweets (19 %).

A. Data Preprocessing

Data preprocessing is often one of the most vital parts of any sentiment analysis project. It involves removing or adding certain data from/to the dataset, removing missing values, or removing noisy data from the set. For sentiment analysis, removing instances with missing values and noisy data from tweets is crucial. Noisy data includes un-encoded emojis, most punctuation, capital letters, and URLs.

It should be noted that all data preprocessing steps, so long as they did not involve fitting and transforming, were carried out identically across the training set and the three testing sets.

B. Noisy Data Preprocessing

Manipulating or removing noisy data from the tweets was the core part of this project's data preprocessing step. Noisy data removal needed to be executed in a specific order, as other orders would disable certain steps from properly removing data. For example, URLs cannot easily be removed from tweets if all punctuation has already been removed (as URLs *contain* punctuation symbols such as `/` or `:`):

1) Emoji decoding: Using the `demoji` library in Python, every emoji, in all tweets, was replaced with a description of that emoji. For example, the emoji 'grinning-face-with-big-eyes' was replaced with the string `:grinning_face_with_big_eyes:`.
2) Then, all uppercase letters were swapped with their lowercase counterparts. Uppercase letters on their own add no sentimental information to a sentence; it typically only signifies proper nouns.
3) URLs, excess white-space, and non-English characters were also identified and removed, as none carry sentiment information relevant to the classification task.
4) Finally, sole instances of punctuation were stripped. Individual punctuation marks do not carry significant sentiment information. However, multiple consecutive instances were retained, as these can imply tone: for example, multiple question marks may indicate confusion or frustration.

All noisy data, other than emojis and non-English characters, were removed using Regular Expressions (RegEx). Non-English characters were removed by using the Python-native ASCII encoder/decoder and flagging it to ignore errors

(which would be raised when non-English characters are encountered).

C. Lemmatisation

A lemma, in morphology, is the canonical or dictionary form of a word. It is the form that is indexed in dictionaries to represent all variations of that word. For example, the lemma for *going* (present progressive), *gone* (past participle), and *went* (simple past) is *go*. There are a large variety of word-forms for every lexeme in a language, which introduces even more noisy data for a sentiment analyser. In practice, from a sentiment point of view, there is not a significant difference between 'the dogs are running happily' and 'the dog run happy'. Both versions retain the same important information: the sentiment of the sentence (positive). The second sentence is obtained after each lexeme undergoes *lemmatisation*. That is, each lexeme in the string is converted into its lemma: 'dogs' $\rightarrow$ 'dog', 'running' $\rightarrow$ 'run', etc.

Lemmatisation is a powerful tool for removing the surrounding noisiness and isolating the important parts of a lexeme, thereby helping with model performance. It further aids with reducing the dimensionality and complexity of the dataset. Fewer word-forms result in a smaller feature space after word-bagging vectorisation is completed (see **Section II-D**), which is more 'digestible' for any model that uses it to train. In this project, it was implemented using the `TextBlob` library in Python. This is a library that provides an intuitive API for the NLTK library. It can be used in, essentially, a single call to both tag various parts of speech (PoS) (nouns, adverbs, verbs, etc.) and lemmatise them. In this project, it was used to tag, and subsequently lemmatise, the following PoSs: nouns, verbs, and adjectives using the `WordNet` corpus.

D. Word-Bagging and N-Grams

The final part of data preprocessing is word-bagging, N-gramming and vectorisation of the data. This is essentially the process used to convert the textual data into a numerical feature space that is suitable for classifiers. This project employed the *Bag-of-Words* (BoW) model, wherein each tweet is represented as a vector over a vocabulary of terms. Though the BoW model disregards word order and syntactic structure, and thereby sequential information, it has been shown to be highly effective for text classification tasks where lexical cues are stronger indicators of class than syntax, semantics, or pragmatics.

Rather than using raw term frequencies, this project applies *Term Frequency–Inverse Document Frequency* (TF-IDF) weighting. TF-IDF assigns higher weights to terms that are frequent within a given document but rare across the corpus, thereby placing more emphasis on informative words but, at the same time, suppressing words that offer little to no lexical meaning (examples being *function words* such as 'the' or 'is'). Additionally, sublinear term frequency scaling is applied, replacing the raw term count $\text{tf}(t, d)$ with $1 + \log(\text{tf}(t, d))$. This dampens the influence of high-frequency terms within individual tweets, which is particularly beneficial for noisy social media text where term repetition (e.g., 'good good good good') is common but not proportionally more informative.

In order to recover some of the semantic information lost by the BoW assumption, N-gram features are incorporated. An N-gram is a sequence of N lexemes: unigrams ($N = 1$) capture individual lexemes (`dog`), bigrams ($N = 2$) capture word-pairs (`dog is`), and trigrams ($N = 3$) capture short phrasal units (`dog is good`). This project uses an N-gram range of $(1, 3)$, meaning that the vocabulary is assumed to include unigrams, bigrams, and trigrams simultaneously. This is vital for sentiment analysis, as negation and intensification are often expressed across multiple lexemes. For example, the bigram 'not good' carries the opposite sentiment to the unigram 'good', and the trigram 'not at all' is a strong negative marker that neither its individual uni- nor bi-grams fully capture.

E. Vector Embeddings

Since the widespread adoption of deep learning approaches to natural language processing, *vector embeddings* (VEs) have become the default method through which lexemes are represented in preprocessing pipelines [4]. Unlike approaches such as the BoW model, VEs are able to retain the syntactic and semantic relationships between word-forms. They are a 'distributed vector representation' for word-forms where each word-form is represented by a comparatively low-dimensional, dense vector that is learned from the specific corpus [4]. In many deep learning approaches, such an embedding acts as the first layer to the model so that the raw lexemes and word-forms may be converted to more useful representations. VEs provide a vital feature: semantically related terms, such as 'run' or 'jog', are clustered closely to each other in the vector representation space, thus retaining the syntactic and semantic relations between words in a document, or tweet, in this case [4].

For this project, the Global Vector (GloVe) 2014 Wikipedia 100-dimensional embeddings were used [5]. An embedding matrix of shape $(|\mathcal{V}|, 100)$ was constructed, where $\mathcal{V}$ is the vocabulary of the 5000 most frequent word-forms in the training corpus. For each word-form in $\mathcal{V}$, if a corresponding GloVe vector existed, it was placed in the correct row of the matrix. Otherwise, the row was left as a zero-vector. This matrix was then loaded into the model's embedding layer as a frozen lookup table. That is, the pre-trained vectors were not updated during training.

Separately, each tweet was converted into a fixed-length sequence of integer indices by mapping each word-form to its position in $\mathcal{V}$, with unknown words mapped to index 0. Sequences shorter than the maximum length (30) were padded with zeros, and sequences longer than the maximum length were truncated. During training, these index sequences are passed into the embedding layer, which looks up the corresponding row of the embedding matrix for each index, producing a dense $(30, 100)$ tensor for each tweet that is then fed into the model. 30 was chosen as the maximum length for a sequence as the 99th percentile of tweet lengths is ~ 29,

meaning that a maximum length of 30 allows the vast majority of tweets to fit into the sequences without requiring truncation and resulting in loss of information.

III. Models & Training

Three models were developed and trained upon this data: two classical algorithms (a linear Support Vector Machine classifier and a logistic regression classifier) and a Long Short-Term Memory deep learning model. This project largely used the `scikit-learn` (SKL) library in Python for the two classical models and data preprocessing. SKL's in-built pipelining tool was used extensively, as were its scoring methods. For the Long Short-Term Memory neural network, the `PyTorch` (PT) library was primarily used in the implementation.

A. Support Vector Machine Classifier – Linear Kernel (SVM-L)

The first model that was developed was the Support Vector Machine using a linear kernel. It was implemented using SKL's `sklearn.svm.LinearSVC(...)` object. Furthermore, `GridSearchCV` was used to perform hyper-parameter tuning for the regularisation parameter C and the class-weights. Tuning over $\mathcal{C} = \{0.5, 1.0, 1.3, 1.5, 2.0\}$ (for $C \in \mathcal{C}$), it was found that $C = 0.5$ provided the optimal $F1$ score value.

Further, the best class-weights were 1.0 for the 'positive' class, 0.5 for the 'neutral' class, and 2.0 for the 'negative' class. Weights were utilised, and more weighting was given to negative and then positive than neutral, because of the fact that tweets of the neutral class are by far more prevalent in the training set than either of the other two. Thus, halving the weight of the neutral class provides a more balanced training procedure.

B. Logistic Regression Classifier (LRC)

For the logistic regression classifier, the `skl.linear_model.LogisticRegression` model was used. The LRC was also trained using `GridSearchCV` for hyper-parameter tuning. The same optimal weights that were found for the SVM-L were used in the LRC. However, it was found that the optimal C for the LRC was $C = 2.0$.

C. Long Short-Term Memory (LSTM) Neural Network Model

A Long Short-Term Memory (LSTM) neural network is an improved variation of the Recurrent Neural Network (RNN) architecture designed to capture dependencies in sequential data over a longer period of time. It solves these limitations of the RNN by the use of a 'memory cell' to store data.

In this project, the LSTM was implemented largely using PyTorch with the following architecture:

- A frozen embedding layer with a dimensionality of 100;
- Two hidden layers of 256 neurons each; and
- An output layer of three neurons – as there are three class labels.

The LSTM also uses the embedding matrix created earlier as model weights.

The model was trained using the ADAM optimiser on the `torch.optim.lr_scheduler.ReduceLROnPlateau` scheduler, which will begin to exponentially decay the learning rate $\alpha = 0.0001$ as the training losses begin to plateau. Furthermore, the `CrossEntropyLoss` was used as a loss function with the same class weights as with the other models: 1.0 for the 'positive' class, 0.5 for the 'neutral' class, and '2.0' for the negative class.

The model was initially trained on 1000 epochs, but this was later reduced to 400 after a clear plateau in training loss was observed shortly before epoch 400. **Figure 1** shows the training loss of the LSTM while being trained on 400 epochs.

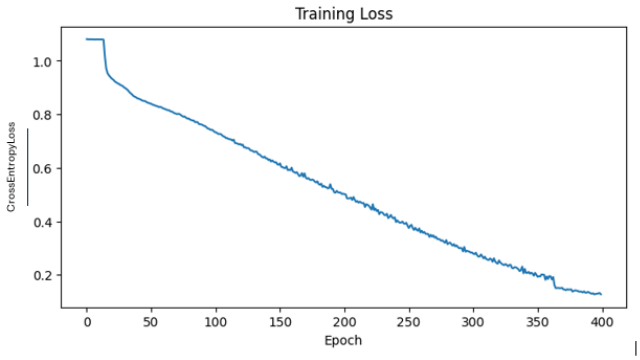


Fig. 1: LSTM Training Losses on 400 Epochs

IV. Testing

As stated in **Section II**, there are three testing sets. Each one was used, in turn, to test all three models, thereby obtaining three sets of results for each model. Metrics were computed using SKL's `skl.metrics.classification_report(...)` function which returns a table of the *precision*, *recall*, and *F1* for each class label (the sentiment), and for the overall dataset.

Emphasis was placed on the macroaveraged (MA) F1 score in this project. For all models, all F1 scores shown were macroaveraged over only the positive and negative classes, not the neutral class, unless specified otherwise.

V. Results & Analysis

A. SVM-L Results

TABLE I: Classification Reports for Test Sets 1, 2, and 3

Set	Metric	Precision	Recall	F1-score	Support
Set 1	accuracy			0.66	3531
	macro avg	0.64	0.63	0.63	3531
Set 2	accuracy			0.67	1853
	macro avg	0.63	0.62	0.63	1853
Set 3	accuracy			0.63	2379
	macro avg	0.59	0.60	0.59	2379

The results for the SVM are shown in **Table I**. The SVM, with a linear kernel, obtained macroaverage F1 scores of 0.59 or 0.63 for each testing set. Further, it yielded average macroaverage scores of approximately 0.62 for both precision and recall across all three datasets. Across all datasets, metrics only differ by exactly 0.01, and neither consistently scores 0.01 higher than the other, either.

Thus, the confusion matrix are examined to provide further insight. **Tables II, III, IV** show the confusion matrices for each testing set for the SVM classifier. The columns are the predicted determinations for that sentiment, and the rows are the actual determinations.

For the negative sentiment, across all matrices, it can be seen that, consistently, approximately a third of the negative tweets in all sets were misclassified as *neutral* (35%, 28%, and 36%, respectively). Adding to this, in all test sets, many neutral tweets were misclassified as *positive* ones: 26%, 35%, and 22%, respectively. Although these proportions are less than the misclassification of negative tweets, they are still substantial. These consistent incorrect predictions, which result in the low precision scores and thereby low macroaverage F1 score, could be caused by bias in the training data. Indeed, recalling from **Section II**, the dataset is composed of 46% neutral tweets and only 19% negative tweets.

However, it is further worth noting that this error pattern appears to be 'directional': negative tweets are misclassified as neutral, and neutral as positive, while relatively few negative or positive tweets are misclassified as the other. This suggests that the primary cause is not class imbalance alone, but poor separability in the feature space at adjacent sentiment boundaries — negative and neutral tweets likely share lexical features, as do neutral and positive ones, while negative and positive are more lexically distinct. The class imbalance in the training data compounds this by providing fewer negative examples from which to learn the negative–neutral boundary.

TABLE II: SVM multiclass confusion matrix for Test Set 0 ($N = 3531$).

	Pred. Neg	**Pred. Neu**	**Pred. Pos**
Act. Neg	281	195	81
Act. Neu	115	996	393
Act. Pos	82	334	1054

TABLE III: SVM multiclass confusion matrix for Test Set 1 ($N = 1853$).

	Pred. Neg	**Pred. Neu**	**Pred. Pos**
Act. Neg	107	57	38
Act. Neu	48	388	233
Act. Pos	35	203	744

B. LRC Results

From **Table V**, it can be seen that, for all macroaverage values, the score never falls below 0.6. All MA F1 scores are either 0.61 or 0.64, providing an average of 0.62. The MA scores for each precision and recall are consistent as well, with each metric yielding between 0.61 and 0.64, regardless of the dataset used.

TABLE IV: SVM multiclass confusion matrix for Test Set 2 ($N = 2379$).

	Pred. Neg	**Pred. Neu**	**Pred. Pos**
Act. Neg	178	132	53
Act. Neu	118	652	213
Act. Pos	93	282	658

Tables VI, VII, and VIII show the confusion matrices for the LRC. The error patterns are similar to those observed for the SVM-L: misclassification is concentrated at adjacent sentiment boundaries, with negative tweets most frequently misclassified as neutral, and neutral as positive. However, the LRC classifies notably more negative tweets correctly than the SVM-L (e.g., 328 vs. 281 in Test Set 0), at the cost of increased neutral-to-positive misclassification (522 vs. 393). A detailed comparison of the two classifiers is provided in **Section V-D**.

TABLE V: Classification Reports for Test Sets 1, 2, and 3

Set	**Metric**	**Precision**	**Recall**	**F1-score**	**Support**
Set 1	accuracy			0.65	3531
	macro avg	0.64	0.64	0.64	3531
Set 2	accuracy			0.66	1853
	macro avg	0.62	0.62	0.61	1853
Set 3	accuracy			0.63	2379
	macro avg	0.61	0.63	0.61	2379

Tables VI, VII, and VIII show the confusion matrices for each dataset, as before.

TABLE VI: LRC multiclass confusion matrix for Test Set 0 ($N = 3531$).

	Pred. Neg	**Pred. Neu**	**Pred. Pos**
Act. Neg	328	144	85
Act. Neu	134	848	522
Act. Pos	108	236	1126

TABLE VII: LRC multiclass confusion matrix for Test Set 1 ($N = 1853$).

	Pred. Neg	**Pred. Neu**	**Pred. Pos**
Act. Neg	116	40	46
Act. Neu	63	314	292
Act. Pos	44	146	792

C. LSTM Results

From **Table IX**, the LSTM yields the lowest performance of all three classifiers, with MA F1 scores ranging from 0.55 to 0.60. Unlike the traditional classifiers, which maintain stable performance across test sets (varying by at most 0.04), the

TABLE VIII: LRC multiclass confusion matrix for Test Set 2 ($N = 2379$).

	Pred. Neg	Pred. Neu	Pred. Pos
Act. Neg	229	81	53
Act. Neu	162	559	262
Act. Pos	114	199	720

TABLE XII: LSTM multiclass confusion matrix for Test Set 2 ($N = 2379$).

	Pred. Neg	Pred. Neu	Pred. Pos
Act. Neg	130	192	41
Act. Neu	93	690	200
Act. Pos	71	376	586

LSTM exhibits a clear downward trend from Test Set 1 (0.60) to Test Set 3 (0.55), suggesting poor generalisation beyond the training distribution.

The confusion matrices (**Tables X, XI, XII**) reveal a significant pattern of neutral-misclassification: 47% of negative tweets and 35% of positive tweets are misclassified as neutral in Test Set 0. Negative recall is particularly poor, with the LSTM recovering fewer than half of truly negative tweets across all test sets. This is likely due to the frozen GloVe embeddings, as they were trained on 2014 Wikipedia which is a very different domain to Twitter, with significantly different vocabulary. Furthermore, capping the vocabulary at 5000 word-forms excludes many informative terms, further limiting classification ability. Additionally, the LSTM lacks explicit N-gram features that aid in encoding negation patterns, and must instead learn such features from context, which proves insufficient given the limited training signal.

TABLE IX: LSTM Classification Reports for Test Sets 1, 2, and 3

Set	Metric	Precision	Recall	F1-score	Support
Set 1	accuracy			0.62	3531
	macro avg	0.63	0.58	0.60	3531
Set 2	accuracy			0.62	1853
	macro avg	0.59	0.55	0.56	1853
Set 3	accuracy			0.59	2379
	macro avg	0.57	0.54	0.55	2379

TABLE X: LSTM multiclass confusion matrix for Test Set 0 ($N = 3531$).

	Pred. Neg	Pred. Neu	Pred. Pos
Act. Neg	236	261	60
Act. Neu	90	1064	350
Act. Pos	60	510	900

TABLE XI: LSTM multiclass confusion matrix for Test Set 1 ($N = 1853$).

	Pred. Neg	Pred. Neu	Pred. Pos
Act. Neg	71	100	31
Act. Neu	35	452	182
Act. Pos	31	325	626

D. Comparative Analysis

The SVM-L and LRC yield very similar MA F1 scores, typically differing by only 0.01–0.02. This is unsurprising, as both are linear classifiers operating in the same TF-IDF feature space. However, their confusion matrices reveal a meaningful trade-off: the LRC classifies more negative tweets correctly (e.g., 328 vs. 281 in Test Set 0) at the cost of increased neutral-to-positive misclassification. This difference is likely a result of their loss functions. The SVM's hinge loss assigns zero loss to correctly classified samples beyond a specific margin, so well-classified negative tweets exert no influence on the boundary. The LRC's log-loss penalises all samples proportionally to confidence, making it more responsive to class weights on the minority negative class.

The LSTM underperforms compared to both traditional classifiers by 0.05–0.08 MA F1, and its performance degrades across test sets more than either linear model, indicating poorer generalisation. Without explicit N-gram features, the LSTM must learn patterns such as negation and intensification from only semantic context. This is a harder task with frozen embeddings and a small 5000-word-form vocabulary.

All three models likely suffer from domain mismatch: GloVe was trained on formal Wikipedia prose, while the target data is informal Twitter text with slang, hashtags, and non-standard orthography. This affects the LSTM most directly, but also degrades lemmatisation and vocabulary coverage for the traditional classifiers. A domain-adapted model such as BERTweet would likely mitigate this substantially.

VI. Conclusion

This report presented three sentiment classifiers for Twitter data: a linear SVM, a logistic regression classifier, and an LSTM with frozen GloVe embeddings. The two traditional classifiers achieved similar MA F1 scores of approximately 0.60–0.64 across all test sets, with the LRC demonstrating a slight advantage in negative sentiment recall due to the properties of its log-loss function. The LSTM underperformed both, achieving 0.55–0.60, with non-negligibly poorer generalisation across test sets.

All three models struggled most with the negative class, likely a consequence of both class imbalance in the training data and poor separability at adjacent sentiment boundaries. Future work could address these limitations through domain-adapted pre-trained models such as BERTweet, which would provide Twitter-specific representations and likely improve performance across all sentiment classes.

References

[1] A. Rajput, "Chapter 3 - natural language processing, sentiment analysis, and clinical analytics," in *Innovation in Health Informatics*, ser. Next Gen Tech Driven Personalized MedSmart Healthcare, M. D. Lytras and A. Sarirete, Eds. Academic Press, 2020, pp. 79–97. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780128190432000034

[2] M. Farhadloo and E. Rolland, *Fundamentals of Sentiment Analysis and Its Applications*. Cham: Springer International Publishing, 2016, pp. 1–24. [Online]. Available: https://doi.org/10.1007/978-3-319-30319-2$_1$

[3] S. Rosenthal, N. Farra, and P. Nakov, "SemEval-2017 task 4: Sentiment analysis in Twitter," in *Proceedings of the 11th International Workshop on Semantic Evaluation*, ser. SemEval '17. Vancouver, Canada: Association for Computational Linguistics, August 2017.

[4] L. Galke, A. Saleh, and A. Scherp, "Word embeddings for practical information retrieval," in *INFORMATIK 2017*, M. Eibl and M. Gaedke, Eds. Bonn: Gesellschaft für Informatik, 2017, pp. 2155–2167.

[5] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543. [Online]. Available: http://www.aclweb.org/anthology/D14-1162